

Scripting quantification

Using NIST DTSA-II

Introduction

- This presentation describes converting spectra into measures of composition
- The presentation uses the data K412 data in [DemoSpectra.zip](#)
- It assumes a detector defined as shown on the next slide
- The scripting syntax is Python 2.5

Status

Enable detector ☒

Name

Detector name

Import

Import from spectrum

Window

Window

Position

Elevation angle °

Azimuthal angle °

Optimal working distance mm

Sample-to-detector distance mm

Crystal parameters

Detector Area mm²

Gold layer nm

Aluminum layer nm

Dead layer μm

Thickness mm

Configuration

Number of channels channels

Zero strobe discriminator eV

Base Performance

Energy scale eV/channel

Zero offset eV

Resolution eV at Mn Ka

Loading spectra

- Spectra may either be:
 - in memory

```
1> ls()  
Name      Spectrum  
s1         Si std  
s2         MgO std  
s3         CaF2 std  
s4         Fe std  
s5         K411 std  
s6         K412 std  
s7         Al2O3 std
```

Variable name to use scripting

I read these in using File-Open.

- read from disk

Load K411 and call it "spec"

```
6> spec = readSpectrum("C:\\Users\\nritchie.NIST\\Desktop\\K411-K412\\K411 std.msa")  
7> spec  
K411 std  
8> display(spec)
```

Display spec in the spectrum display window.

Preparing spectra

- Standards and unknown spectra must contain the following data: “Live time”, “Probe current” and/or “Probe current (after)”, “Elevation”, “Beam energy”, “Energy scale”, “Energy offset” and “Standard composition”.
- Reference spectra don’t require probe current or live time.

Live time:	epq.SpectrumProperties.LiveTime
Probe current:	epq.SpectrumProperties.FaradayBegin
Probe current (end):	epq.SpectrumProperties.FaradayEnd
Elevation:	epq.SpectrumProperties.Elevation
Beam energy:	epq.SpectrumProperties.BeamEnergy
Energy scale:	epq.SpectrumProperties.EnergyScale
Energy offset:	epq.SpectrumProperties.EnergyOffset
Detector:	epq.SpectrumProperties.Detector

```
sp=s1.getProperties()  
sp.setNumericProperty(epq.SpectrumProperties.FaradayBegin, 2.5)  
sp.getNumericProperty(epq.SpectrumProperties.FaradayBegin)
```

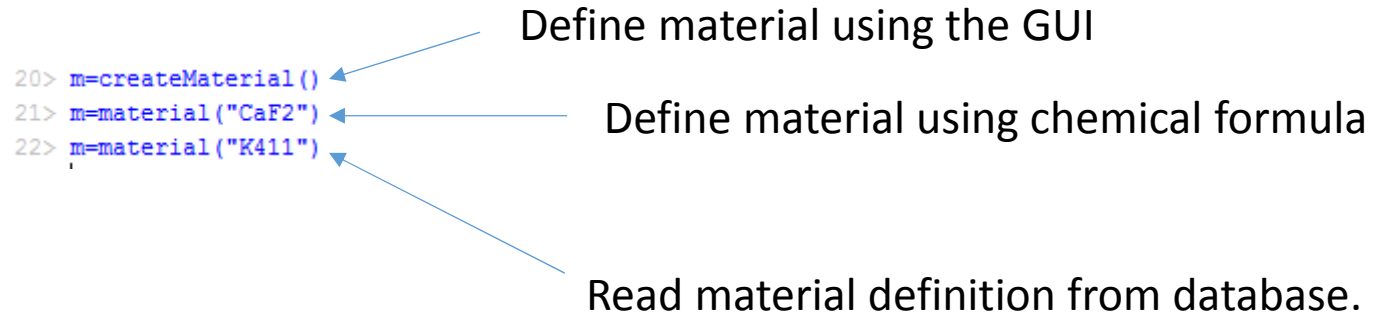
Setting the detector

The variable name used to identify detectors in scripts.

```
12> listDetectors()
Name      Detector
d10       EDAX
d17       PulseTor (Best)
d13       SDD 3.2 µs
d6        Inca 80 PP=4, 10 eV/ch
d31       Oxford X-Max 80 mm²
d26       Detector 0 - Medium
d1        Si(Li)
d16       SDD (Medium, 2048)
d5        80 mm²
d30       Microcal Clone
d7        Inca 80 PP=4, 5 eV/ch
```

```
sp = s1.getSpectrumProperties()
sp.setDetector(d2)
```

Setting the standard composition



20> `m=createMaterial()` ← Define material using the GUI
21> `m=material("CaF2")` ← Define material using chemical formula
22> `m=material("K411")` ← Read material definition from database.

Set the “standard composition” of the spectrum s1 to pure Si.

```
23> s1.getProperties().setCompositionProperty(epq.SpectrumProperties.StandardComposition, material("Si"))
```

The primary method for scripting the quantification of EDS spectra is the “quantify” method. Quantify takes spectra objects for the unknown, standard and references and returns

```
13> help(quantify)
quantify(unk, stds, refs={}, preferred=(), elmByDiff = None, oByStoic=False, oxidizer=None)
    Quantify an unknown spectrum against a set of standard spectra.  You can optionally provide a set of references to use for shape
    information.
+ unknown - An ISpectrumData derived object with EDSDetector defined
+ stds - A dictionary mapping epq.Element into an ISpectrumData derived object
+ refs - (optional) A dictionary mapping an x-ray transition or x-ray transition set into an ISpectrumData object
+ preferred - (optional) A collection of XRayTransition objects specifying the preferred line for quantification
+ elmByDiff - (optional) Element to compute by difference from 100%
+ oByStoic - (optional) True to calculate oxygen by stoichiometry, False otherwise
+ oxidizer - (optional) An epq.Oxidizer object to use in place of the default set
```

Returns a epq.QuantifyUsingStandards.Result object.

```
10> quantify(s6, {"Si":s1, "Mg":s2, "Ca":s3, "Fe":s4, "Al":s7, "O":s7})
gov.nist.microanalysis.EPQLibrary.QuantifyUsingStandards$Result@1797469c
11> res=quantify(s6, {"Si":s1, "Mg":s2, "Ca":s3, "Fe":s4, "Al":s7, "O":s7})

20> tmp=res.getComposition()
21> print tmp.descriptiveString(False)
K412 std = [O(0.4556±0.0200 mass frac),Mg(0.1161±0.0005 mass frac),Al(0.0479±0.0002 mass frac),Si(0.2129±0.0005 mass frac),Ca(0.1081±0.
0004 mass frac),Fe(0.0784±0.0008 mass frac),Σ=1.0188±0.0210]
```


O by stoichiometry

```
11> res=quantify(s6, stds={"Si":s1, "Mg":s2, "Ca":s3, "Fe":s4, "Al":s7}, refs={"O K-L3":s7}, oByStoic=True)
12> print res.getComposition().descriptiveString(False)
K412 std = [O(0.4207±0.0007 mass frac), Mg(0.1133±0.0005 mass frac), Al(0.0469±0.0002 mass frac), Si(0.2098±0.0005 mass frac), Ca(0.1076±0.0004 mass frac), Fe(0.0782±0.0008 mass frac), Σ=0.9764±0.0015]
```

The default stoichiometries as defined in epq.Oxidizer are used unless a custom oxidizer is specified.

```
2> ox=epq.Oxidizer()
3> dir(ox)
['OXYGEN', '__class__', '__copy__', '__deepcopy__', '__delattr__', '__doc__', '__eq__', '__getattribute__', '__hash__', '__init__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__str__', '__unicode__', 'class', 'compute', 'equals', 'getClass', 'getComposition', 'getOxidationState', 'hashCode', 'notify', 'notifyAll', 'setOxidizationState', 'toHTML', 'toHTMLTable', 'toHTMLTable2', 'toOxideFraction', 'toString', 'toTable', 'wait']
4> ox.getOxidationState(epq.Element.Fe)
2
5> ox.getOxidationState(epq.Element.O)
-2
```

This works from Halley 30-Apr-2014 release on...

Quantify multiple spectra

Saves processor time by eliminating some redundant calculations when the same standards, refs, etc are used to process a set of unknowns.

```
1> help(multiQuant)
multiQuant(det, e0, stds, refs={}, preferred={}, oByStoic=False, oxidizer=None)
Configure a QuantifyUsingStandards object which can be then used to quantifying multiple spectra.
Example:
    qus=multiQuant(det,e0,stds,refs)
    for spec in map:
        res=qus.compute(unknown)
        print res.getComposition()

+ det - The epq.EDSDetector
+ e0 - The beam energy in keV
+ stds - A dictionary mapping epq.Element into an ISpectrumData derived object
+ refs - (optional) A dictionary mapping an x-ray transition or x-ray transition set into an ISpectrumData object
+ preferred - (optional) A collection of XRayTransition objects specifying the preferred line for quantification
+ elmByDiff - (optional) Element to compute by difference from 100%
+ oByStoic - (optional) True to calculate oxygen by stoichiometry, False otherwise
+ oxidizer - (optional) An epq.Oxidizer object to use in place of the default epq.Oxidizer()
```

Composition from k-ratios

Maybe you know the k-ratios and want to compute the composition.
(Maybe from WDS..)

```
17> help(quant)
quant(kratios, toa = 40.0, e0 = 15.0, stds = {}, elmByDiff = None, elmByStoic = None, assumedStoic = {}, extraProps = None)
Performs a quantitative correction on the specified dictionary of kratios = { tr0 : kr}
+ kratios = { tr0 : kr0, tr1 : kr1, ..., trn : krn } where trX = epq.XRayTransition or epq.XRayTransition
  + XRayTransition can be constructed using 'transition(...)'
  + XRayTransitionSet can be constructed using 'getTransitionSet(elm, family)'
+ e0 - Beam energy in keV (or an IXRayDetector object)
+ toa - Take off angle in degrees
+ std = { elm0 : comp0, elm1 : comp1, ..., } where elmX is an element and compX is the associated composition
  + Missing elements are assumed to be pure
+ elmByDiff - Element to compute as a difference from 100%
+ oByStoic - (optional) True to calculate oxygen by stoichiometry, False otherwise
+ oxidizer - (optional) An epq.Oxidizer object to use in place of the default epq.Oxidizer()
+ xtraProps - epq.SpectrumProperties() object with extra properties defined
```